

以下均为与deepseek R1模型的交互：

```
Q1: def low_pass_filter(shape, radius=100): h, w = shape cy, cx = h//2, w//2 y = torch.arange(h).reshape(-1,1) x = torch.arange(w).reshape(1,-1) mask = ((y-cy)**2 + (x-cx)**2) < radius**2 # 圆形区域判断 return mask.float()
```

```
mask = low_pass_filter(fft.shape, radius=0.1) mask2 = 1 - mask low_pass_filtered_fft = fft * mask low_pass_filtered_ifft = torch.abs(torch.fft.ifft2(torch.fft.ifftshift(low_pass_filtered_fft))) high_pass_filtered_fft = fft * mask2 high_pass_filtered_ifft = torch.abs(torch.fft.ifft2(torch.fft.ifftshift(high_pass_filtered_fft)))
```

```
shaper_fft = fft + fft * mask2
```

```
shaper_ifft = torch.abs(torch.fft.ifft2(shaper_fft))
```

```
plt.subplot(1, 3, 1) plt.imshow(low_pass_filtered_ifft.numpy(), cmap='gray') plt.imshow(high_pass_filtered_ifft.numpy(), cmap='gray') plt.imshow(shaper_ifft.numpy(), cmap='gray') plt.savefig('result/task3_low_pass_filtered.png', low_pass_filtered_ifft.numpy(), cmap='gray') plt.savefig('result/task3_high_pass_filtered.png', high_pass_filtered_ifft.numpy(), cmap='gray') plt.savefig('result/task3_shaper_filtered.png', shaper_ifft.numpy(), cmap='gray')
```

```
plt.subplot(1, 3, 3)
```

```
plt.imshow(shaper_ifft.numpy(), cmap='gray')
```

```
plt.savefig('result/task3_shaper_filtered.png', shaper_ifft.numpy(), cmap='gray')
```

plt.show()为什么这样的参数下 高通滤波的结果呈现出反色的效果 黑色变成了白色

A1:在高通滤波后出现反色效果（黑色变白色）的根本原因在于 **高频分量对图像亮度均值的破坏** 和 **显示时的自动归一化处理**。以下是分步解释：

一、滤波过程的数学本质

1. 低通滤波：

- 保留以频谱中心（低频区域）为主的频率分量。
- 包含 **直流分量 (DC Component)**，即图像的平均亮度。
- 恢复后的图像亮度分布与原图一致。

2. 高通滤波：

- 抑制中心低频分量（包括直流分量），保留外围高频分量。
 - 失去平均亮度（所有像素值的均值为零）。
 - 恢复的图像在数值上以零为中心波动（正负交替）。
-

二、反色现象的直接原因

1. 直流分量的丢失

- 高通滤波后，图像的像素值均值为零，高频分量表现为正负交替的波动。
- 当通过 `torch.abs()` 取绝对值时，所有负值被转换为正值，导致整体亮度提升。

2. 显示归一化的副作用

- 默认情况下，`plt.imshow()` 会自动将数据归一化到 `[0, 1]`（或当前数据的最小最大值）。
- 高通滤波后的图像数据范围较小（接近零附近），归一化后：
 - 原图中的深色区域（接近0）被拉伸到归一化范围的最低端（显示为白色）。
 - 高频波动中的较高值被显示为深色。

三、实验验证

1. 检查高通滤波后的数值范围

```
print("高通滤波后像素值范围: ",
      high_pass_filtered_ifft.min().item(),
      "~",
      high_pass_filtered_ifft.max().item())
```

输出示例：

```
高通滤波后像素值范围:  0.0 ~ 0.02
```

- 数值范围极小（接近零），而原图范围可能是 `0.0 ~ 1.0`。

2. 禁用归一化显示

```
plt.imshow(high_pass_filtered_ifft.numpy(),
           cmap='gray',
           vmin=0,
           vmax=1) # 固定显示范围为原图范围
```

- 此时图像会显示为全黑（因实际值远小于1）。

3. 数值放大观察细节

```
high_pass_scaled = high_pass_filtered_ifft * 50 # 放大50倍
plt.imshow(high_pass_scaled.numpy(), cmap='gray')
```

- 可以看到实际的高频边缘信息（但需大幅提升对比度）。

四、反色效果的形成机制

步骤	数据状态	显示效果
高通滤波 + 绝对值	数值集中在 $[0, \epsilon]$	原始范围归一化后接近全白
原图	数值分布在 $[0, 1]$	正常灰度分布
显示自动归一化	将 $[0, 0.02]$ 映射到 $[0, 1]$	低值被拉伸为白色

- **关键点：**高通滤波后的极小值范围被错误地解释为高亮度。

五、解决方案

1. 恢复亮度均值

```
# 在频域中保留少量低频分量（半高通滤波）
mask2 = 1 - mask * 0.99 # 保留1%的低频信息
high_pass_filtered_fft = fft * mask2
```

- 保留少量直流分量，避免像素均值归零。

2. 手动调整显示范围

```
plt.imshow(high_pass_filtered_ifft.numpy(),
            cmap='gray',
            vmin=0,
            vmax=0.1) # 根据实际值范围调整
```

3. 后处理增强对比度

```
high_pass_normalized = (high_pass_filtered_ifft - high_pass_filtered_ifft.min()) / \
                        (high_pass_filtered_ifft.max() -
 high_pass_filtered_ifft.min())
plt.imshow(high_pass_normalized.numpy(), cmap='gray')
```

六、总结

- **反色假象：**因高通滤波后数值范围过小，显示时被归一化误导。

- **物理本质**：丢失的直流分量导致图像失去全局亮度参考。
- **正确理解**：高通滤波结果本应显示边缘和纹理（深色背景+亮边），但因数值压缩导致视觉反转。

Q2:说明fftshift背后的数学原理 然后说明为什么进行傅立叶变换时用了fftshift后要配对着使用ifftshift A2:###
fftshift的数学原理

fftshift 的作用是将傅里叶变换后的频谱数据进行**循环移位**，使得低频分量（即能量集中的区域）从默认的四角位置移动到频谱中心。其数学本质是通过**重新排列频域数据**，调整坐标系的对称性，具体步骤如下：

1. 傅里叶变换的默认排列：

- 假设原始信号是实信号（如图像），其傅里叶变换后的频谱具有共轭对称性。
- 未移位时，低频分量（直流分量和低频波）分布在频谱四角，高频分量（边缘和细节）分布在中心。

2. 循环移位的操作：

- 对于二维频谱矩阵，假设尺寸为 (H, W) ，**fftshift** 会将频谱沿高度方向循环移动 $H/2$ ，沿宽度方向循环移动 $W/2$ 。
- 数学表达式为：
$$[\text{fftshift}(F)(u, v) = F\left((u + H/2) \bmod H, (v + W/2) \bmod W\right)]$$
 其中 F 是原始频谱， u 和 v 是坐标。

3. 物理意义：

- 将零频率分量（DC分量）移到频谱中心，便于观察和滤波操作。
- 符合人类对“低频在中心”的直观认知（如常见的热图显示）。

为什么需要配对使用 **fftshift** 和 **ifftshift**？

1. 恢复频域坐标系的必要性

- 傅里叶变换（FFT）和逆变换（IFFT）的数学定义依赖于固定的坐标系。
- **fftshift 改变了坐标系**：若直接对移位后的频谱进行逆变换，会导致频域相位偏移，从而破坏时域信号的结构。

2. 数学可逆性要求

- **fftshift** 和 **ifftshift** 是严格互逆的操作：
$$[\text{ifftshift}(\text{fftshift}(F)) = F]$$
- 若省略 **ifftshift**，逆变换的输入频谱坐标系错误，会导致恢复的信号出现**相位反转**或**空间错位**。

3. 实验验证

- **不配对使用的后果**：

```
# 错误操作示例
fft_shift = torch.fft.fftshift(fft)
recon_wrong = torch.fft.ifft2(fft_shift).real # 未使用 ifftshift
```

此时恢复的图像会出现以下问题：

- **棋盘格效应**：像素值正负交替（相位偏移导致）。
- **能量中心偏离**：图像内容可能错位。
- **正确操作：**

```
# 正确恢复流程
fft_restored = torch.fft.ifftshift(fft_shift)
recon_correct = torch.fft.ifft2(fft_restored).real
```

配对使用的深层原因

1. 相位一致性的保证

- 傅里叶变换的相位谱对时域信号的位置极其敏感。
- `fftshift` 改变了频域坐标，相当于对时域信号引入了全局相位偏移： $[\mathcal{F}^{-1}(\text{fftshift}(F)) = (-1)^{x+y} \cdot f(x, y)]$ 其中 $(-1)^{x+y}$ 是相位偏移因子，导致像素值符号交替。
- `ifftshift` 消除这一偏移，恢复原始相位。

2. 奇偶尺寸的处理

- 当图像尺寸为奇数时，`fftshift` 和 `ifftshift` 的位移量不同：
 - `fftshift` 的位移量为 $(H//2, W//2)$ （向下取整）。
 - `ifftshift` 的位移量为 $((H+1)//2, (W+1)//2)$ 。
- 若混用两者，奇尺寸图像的恢复会失败。

3. 频域滤波的准确性

- 在频域进行滤波（如低通滤波）时，滤波器掩膜必须与频谱坐标系对齐。
- 若未通过 `ifftshift` 恢复坐标系，滤波操作会作用在错误的频域区域。

操作流程的黄金法则

1. 正向变换流程：

```
fft = torch.fft.fft2(img)          # 原始频谱（低频在四角）
fft_shift = torch.fft.fftshift(fft) # 低频移到中心
```

2. 逆向变换流程：

```
fft_restored = torch.fft.ifftshift(fft_shift) # 恢复原始坐标系
recon = torch.fft.ifft2(fft_restored).real    # 正确恢复时域信号
```

总结

操作	目的	数学本质
<code>fftshift</code>	将低频分量移到频谱中心，便于分析	循环移位坐标系
<code>ifftshift</code>	恢复原始坐标系，保证逆变换正确性	逆向循环移位
配对使用	确保频域-时域映射的严格可逆性	消除相位偏移和坐标错位

核心结论：`fftshift` 和 `ifftshift` 是傅里叶变换中不可分割的互补操作，前者用于可视化与滤波设计，后者用于恢复数学一致性。忽略配对使用会导致不可预测的相位错误和图像失真。

Q3: $F^{-1}(\text{fftshift}(F))(x, y) = (-1)^{x+y} f(x, y)$ 详细推导这个公式 A3: 要推导公式

$F^{-1}(\text{fftshift}(F))(x, y) = (-1)^{x+y} f(x, y)$ ，我们需要分析傅里叶变换的移位性质及其对时域信号的影响。

1. 傅里叶变换的移位性质

对于二维离散傅里叶变换（DFT），时域中的平移操作会导致频域中的相位变化。具体来说：

- 若时域信号 $f(x, y)$ 的傅里叶变换为 $F(u, v)$,
- 则时域平移后的信号 $f(x - a, y - b)$ 的傅里叶变换为： $[F(u, v) \cdot e^{-j2\pi \left(\frac{a}{M} u + \frac{b}{N} v \right)}]$

2. `fftshift` 的数学操作

`fftshift` 将频谱沿两个方向循环移位 $\frac{M}{2}$ 和 $\frac{N}{2}$ 点，使得低频分量从四角移动到中心。数学上，移位后的频谱为： $[F'(u, v) = F\left(u + \frac{M}{2}, v + \frac{N}{2}\right) \bmod M, N]$

3. 逆变换的相位偏移

对移位后的频谱 $F'(u, v)$ 进行逆傅里叶变换： $[g(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F'(u, v) e^{j2\pi \left(\frac{u}{M} x + \frac{v}{N} y \right)}]$ 通过变量替换 $u' = (u + M/2) \bmod M$ 和 $v' = (v + N/2) \bmod N$ ，可得： $[g(x, y) = \frac{1}{MN} \sum_{u'=0}^{M-1} \sum_{v'=0}^{N-1} F(u', v') e^{j2\pi \left(\frac{(u' - M/2)x}{M} + \frac{(v' - N/2)y}{N} \right)}]$ 展开指数项： $[e^{j2\pi \left(\frac{u'x}{M} - \frac{x^2}{2M} + \frac{v'y}{N} - \frac{y^2}{2N} \right)} = e^{j2\pi \frac{u'x}{M}} \cdot e^{j2\pi \frac{v'y}{N}} \cdot e^{-j\pi \left(\frac{x^2}{M} + \frac{y^2}{N} \right)}]$ 由于 $e^{-j\pi(x+y)} = (-1)^{x+y}$ ，因此： $[g(x, y) = (-1)^{x+y} \cdot \underbrace{\frac{1}{MN} \sum_{u'=0}^{M-1} \sum_{v'=0}^{N-1} F(u', v') e^{j2\pi \left(\frac{u'x}{M} + \frac{v'y}{N} \right)}}_{f(x, y)}]$ 最终得到： $[g(x, y) = (-1)^{x+y} f(x, y)]$

4. 关键结论

- 相位偏移的来源：**`fftshift` 在频域的循环移位等价于在时域引入相位因子 $(-1)^{x+y}$ 。
- 恢复原信号的步骤：**必须使用 `ifftshift` 将频谱移回原位，以消除相位偏移： $[F^{-1}(\text{ifftshift}(\text{fftshift}(F))) = f(x, y)]$

数学推导总结

步骤	操作	数学表达式
1. fftshift	频域循环移位 $M/2$ 和 $N/2$	$F'(u, v) = F((u + M/2) \bmod M, (v + N/2) \bmod N)$
2. 逆变换	计算 $F^{-1}(F')$	$g(x, y) = \sum F' \cdot e^{j2\pi(ux/M + vy/N)}$
3. 变量替换	$u' = u + M/2,$ $v' = v + N/2$	$g(x, y) = (-1)^{x+y} \sum F(u', v') e^{j2\pi(u'x/M + v'y/N)}$
4. 结果	相位因子出现	$g(x, y) = (-1)^{x+y} f(x, y)$

物理意义

- **棋盘格效应**: 未恢复移位时, 逆变换结果的像素值符号交替变化 ($(-1)^{x+y}$), 导致显示为棋盘格模式。
- **正确恢复的关键**: `ifftshift` 消除频域移位, 恢复时域信号的原始相位。

此推导严格证明了为何在傅里叶变换中必须配对使用 `fftshift` 和 `ifftshift`。

Q4: \subsection{时域与频域的卷积速率差异} 在这个实验中，我们分别设计三种不同尺寸的输入和卷积核，输入尺寸为100100、200200、300300，卷积核尺寸为33、77、1515。然后我们设计两种卷积方式，一种是在时域中直接利用conv2d卷积，另一种是利用fft2转换到频域相乘后再用ifft2转换回时域，实验得到如下的结果。

```
\begin{table}[H] \centering \captionnamefont{\wuhao\bf\heiti} \captiontitlefont{\wuhao\bf\heiti}
\caption{全线表示例} \label{tab:eg1} \liuhao \begin{tabular}{|c|c|c|c|c|} \hline
```

```

\multicolumn{2}{|c|}{\multirow{2}{\mathit{S}_i}}&\multicolumn{3}{c|}{事件}&\cline{3-5}\multicolumn{2}{|c|}
{}&50&100&150\ \hline \multirow{3}{策略}&50&0&100&200\ \cline{2-5}&100&100&0&100\ \cline{2-5}
&150&200&100&0\ \cline{2-5}&200&300&200&100\ \hline \end{tabular} \end{table}

```

```
\begin{table}[H] \centering \centering \captionnamefont{\wuhao\bf\heiti} \captiontitlefont{\wuhao\bf\heiti}
\caption{全线表示例} \label{tab:eg2} \liuhao \begin{tabular}{|c|c|c|c|c|} \hline
```

如何把这两张表并排放起来 A4:在 LaTeX 中, 可以通过 `minipage` 环境将两表并排放置。修改后的代码如下:

```
\begin{table}[H]
\centering
% 左表
\begin{minipage}{0.45\textwidth}
\centering
\captionnamefont{\wuhao\bf\heiti}
\captiontitlefont{\wuhao\bf\heiti}
\caption{全线表示例} \label{tab:eg1}
\liuhao
\begin{tabular}{|c|c|c|c|c|c|}
```

```

\hline
\multicolumn{2}{|c|}{\multirow{2}*{$S_i$}} & \multicolumn{3}{c|}{事件} \\
\cline{3-5}
\multicolumn{2}{|c|}{} & 50 & 100 & 150 \\
\hline
\multirow{3}*{策略} & 50 & 0 & 100 & 200 \\
\cline{2-5}
& 100 & 100 & 0 & 100 \\
\cline{2-5}
& 150 & 200 & 100 & 0 \\
\cline{2-5}
& 200 & 300 & 200 & 100 \\
\hline
\end{tabular}
\end{minipage}
\hfill
% 右表
\begin{minipage}{0.45\textwidth}
\centering
\captionnamefont{\wuhao\bf\heiti}
\captiontitlefont{\wuhao\bf\heiti}
\caption{全线表示例} \label{tab:eg2}
\liuhao
\begin{tabular}{|c|c|c|c|c|}
\hline
\multicolumn{2}{|c|}{\multirow{2}*{$S_i$}} & \multicolumn{3}{c|}{事件} \\
\cline{3-5}
\multicolumn{2}{|c|}{} & 50 & 100 & 150 \\
\hline
\multirow{3}*{策略} & 50 & 0 & 100 & 200 \\
\cline{2-5}
& 100 & 100 & 0 & 100 \\
\cline{2-5}
& 150 & 200 & 100 & 0 \\
\cline{2-5}
& 200 & 300 & 200 & 100 \\
\hline
\end{tabular}
\end{minipage}
\end{table}

```

代码说明：

1. **minipage 环境**：将每个表格包裹在 `minipage` 中，宽度设为 `0.45\textwidth`（留出 10% 的间距）。
2. **\hfill**：在左右 `minipage` 之间插入弹性间距，实现横向撑满。
3. **保持原有格式**：保留原表格的 `\captionnamefont`、`\captiontitlefont` 和 `\liuhao` 等格式设置。
4. **浮动体优化**：由于两表原本都使用 `[H]` 定位，合并后统一用一个 `[H]` 保证位置。

效果：

- 两表将水平并排，间距自动填充
- 每个表格保留独立标题和标签

- 格式与原表格完全一致。

Q5:请问torch.nn.functional.conv1d中的weight参数为什么outchannel不需要/groups inchannel需要/groups torch会自动划分每个group对应的卷积核吗 A5:在PyTorch的torch.nn.functional.conv1d中, weight参数的形状为(out_channels, in_channels/groups, kernel_size)。这种设计源于分组卷积 (Grouped Convolution) 的实现逻辑, 具体原因如下:

1. groups的作用机制

- **输入通道的分组**: 输入通道被均分为groups组, 每组包含in_channels/groups个通道。
- **输出通道的分组**: 输出通道同样被均分为groups组, 每组生成out_channels/groups个通道。
- **独立卷积操作**: 每个组的卷积核仅处理对应组的输入通道, 并生成对应组的输出通道。最终所有组的输出拼接为完整的out_channels。

2. weight参数的形状解析

- **out_channels不需要显式除以groups**
out_channels表示总输出通道数, 而每个组实际负责生成out_channels/groups个通道。PyTorch内部会自动将weight视为groups个独立的子卷积核, 每个子卷积核的通道数为out_channels/groups。
- **in_channels必须显式除以groups**
每个组的卷积核只能看到in_channels/groups个输入通道。因此, weight的第二维必须明确匹配每个组实际处理的输入通道数。

3. PyTorch的自动分组逻辑

- **weight参数的划分**
当指定groups时, PyTorch会将weight视为由groups个子卷积核组成的张量。每个子卷积核的形状为(out_channels/groups, in_channels/groups, kernel_size)。
- **输入输出的对应关系**
输入的第i组 (in_channels/groups通道) 与weight的第i个子卷积核进行卷积, 生成输出的第i组 (out_channels/groups通道)。最终输出是所有组的拼接结果。

4. 关键约束条件

- **in_channels和out_channels必须均能被groups整除**
否则无法均分通道到各分组, 导致报错。例如:

```
# 正确: in_channels=6, out_channels=8, groups=2
conv = nn.Conv1d(6, 8, kernel_size=3, groups=2) # 6%2=0, 8%2=0

# 错误: in_channels=5, groups=2 (5%2≠0)
```

总结

- **weight的形状**: (`out_channels`, `in_channels/groups`, `kernel_size`)
 - `out_channels`是总输出通道数，无需显式分组（由PyTorch内部处理）。
 - `in_channels/groups`确保每个卷积核仅处理对应分组的输入通道。
- **PyTorch的自动分组**: 通过`groups`参数隐式划分输入、输出通道和`weight`的子卷积核，保证分组卷积的正确执行。

Q6:什么卷积核能去噪 (3*3) A6:在图像处理中，以下3x3卷积核或方法常用于去噪：

1. 均值滤波核

- **核值**:

```
[1/9, 1/9, 1/9]
[1/9, 1/9, 1/9]
[1/9, 1/9, 1/9]
```

- **作用**: 通过均匀平均邻域像素值，减少高斯噪声等随机噪声，但会导致图像模糊。
- **适用场景**: 简单快速去噪，对计算资源要求低。

2. 高斯滤波核

- **核值** (归一化版本) :

```
[1, 2, 1]
[2, 4, 2]
[1, 2, 1]
```

总和为16，需除以16归一化。

- **作用**: 基于高斯函数分配权重，中心像素权重更大，在平滑噪声的同时保留更多边缘信息。
- **适用场景**: 高斯噪声或需保留细节的轻度去噪。

3. 中值滤波 (非卷积操作)

- **方法**: 取3x3邻域内9个像素的中值替换中心像素。
- **作用**: 对椒盐噪声 (突发性黑白噪点) 效果显著，且能避免均值滤波的模糊问题。
- **适用场景**: 处理扫描文档、传感器噪声等椒盐噪声。

其他注意事项

- **噪声类型适配**:
 - **高斯/均匀噪声**: 优先使用均值或高斯滤波。
 - **椒盐噪声**: 选择中值滤波。
- **权衡取舍**: 平滑去噪会损失细节，需根据需求调整滤波强度 (如重复滤波次数或调整核尺寸)。

示例代码 (Python OpenCV)

```
import cv2
import numpy as np

# 读取图像
image = cv2.imread("noisy_image.jpg")

# 均值滤波
mean_kernel = np.ones((3,3), np.float32) / 9
mean_filtered = cv2.filter2D(image, -1, mean_kernel)

# 高斯滤波
gaussian_filtered = cv2.GaussianBlur(image, (3,3), 0)

# 中值滤波
median_filtered = cv2.medianBlur(image, 3)
```

根据具体噪声类型选择合适的滤波方法，可显著提升去噪效果。